

DAA Lab - Experiment 5

Implement Job Sequencing with Deadlines using Greedy Algorithm

Dr. Ayush Kumar Agrawal
Assistant Professor, School of Computer Science
UPES, Dehradun

Objective

- Schedule jobs to maximize profit when each job has a deadline and profit.
- Implement greedy strategy for Job Sequencing.
- Compare with brute-force scheduling.

- Each job has:
 - A deadline (must be completed before or on it).
 - A profit if completed within the deadline.
- Strategy:
 - 1 Sort jobs in descending order of profit.
 - 2 Assign jobs to latest possible available slot before deadline.
 - 3 Skip job if no slot available.
- Greedy choice ensures maximum profit.

Algorithm: Job Sequencing with Deadlines

Job_Sequencing(jobs[], n)

- ① Sort jobs in descending order of profit.
- ② Initialize result array (time slots) as empty.
- ③ For each job i in sorted list:
 - For slot = min(deadline[i], n) downto 1:
 - If slot empty:
 - Assign job[i] to slot.
 - Break.
- ④ Return scheduled jobs and total profit.

C Code - Job Sequencing (Part 1)

```
#include <stdio.h>

int main() {
    int n, i, j, maxDeadline = 0;
    printf("Enter number of jobs: ");
    scanf("%d", &n);

    int profit[n], deadline[n], slot[n];
    char job[n];

    printf("Enter job IDs: ");
    for(i=0; i<n; i++) scanf(" %c", &job[i]);

    printf("Enter profits: ");
    for(i=0; i<n; i++) scanf("%d", &profit[i]);

    printf("Enter deadlines: ");
    for(i=0; i<n; i++) {
        scanf("%d", &deadline[i]);
        if(deadline[i] > maxDeadline)
            maxDeadline = deadline[i];
    }
```

C Code - Job Sequencing (Part 2)

```
// initialize slots
for(i=0; i<maxDeadline; i++)
    slot[i] = -1;

// sort jobs by profit (descending)
for(i=0; i<n-1; i++) {
    for(j=i+1; j<n; j++) {
        if(profit[i] < profit[j]) {
            int tempP = profit[i]; profit[i] = profit[j]; profit[j] = tempP;
            int tempD = deadline[i]; deadline[i] = deadline[j];
            deadline[j] = tempD;
            char tempJ = job[i]; job[i] = job[j]; job[j] = tempJ;
        }
    }
}
```

C Code - Job Sequencing (Part 3)

```
int totalProfit = 0;

// schedule jobs
for(i=0; i<n; i++) {
    for(j=deadline[i]-1; j>=0; j--) {
        if(slot[j] == -1) {
            slot[j] = i;
            totalProfit += profit[i];
            break;
        }
    }
}

printf("Job sequence: ");
for(i=0; i<maxDeadline; i++) {
    if(slot[i] != -1)
        printf("%c ", job[slot[i]]);
}
printf("\nTotal Profit = %d\n", totalProfit);
}
```

Sample Input/Output

Input:

$n = 5$

Jobs = A, B, C, D, E

Profits = 100, 19, 27, 25, 15

Deadlines = 2, 1, 2, 1, 3

Output:

Job sequence: C A E

Total Profit = 142

Complexity Analysis

- Sorting: $O(n^2)$ (in this implementation)
- Job assignment: $O(n \times \text{maxDeadline})$
- Overall: $O(n^2)$
- With efficient sorting + disjoint set: $O(n \log n)$

Observation Table

Jobs	Deadlines	Max Profit
3	2,1,2	127
5	2,1,2,1,3	142
6	1,2,2,1,3,2	180

Comparison

- Greedy gives optimal result for job sequencing.
- Brute force would try all subsets $\rightarrow$ exponential.
- Greedy is efficient for deadlines-based scheduling.

Viva Questions

- 1 Why do we sort jobs by profit first?
- 2 Why do we assign jobs to the latest slot before deadline?
- 3 What is the worst-case time complexity?
- 4 How can Union-Find improve efficiency?

Conclusion

- Job Sequencing with deadlines can be solved optimally using Greedy.
- Sorting jobs by profit and placing in latest available slot ensures maximum profit.
- Complexity mainly depends on sorting.

References

- Cormen et al., *Introduction to Algorithms*.
- Horowitz & Sahni, *Fundamentals of Computer Algorithms*.